Escuela Colombiana de Ingeniería Julio Garavito

Desarrollo Orientado por Objetos (DOPO) 501 — 2026-2

Laboratorio 5

Interfaz gráfica: HungrySnake

Hecho por:

Juan David Espitia
Cristian Camilo Salamanca

Profesora:

María Irma Díaz Rozo

Bogotá D.C., 2026

Índice

1. Parte I. Maqueta **2**

1.1. Directorios 2

1.2. Ciclo 0: Ventana vacía – Salir 2

1.3. Ciclo 1: Ventana con menú – Salir 5

1.4. Ciclo 2: Salvar y abrir 8

1.5. Ciclo 3: Forma de la ventana principal 11

1.6. Ciclo 4: Cambiar colores 14

2. Parte II. Funcionalidades básicas **16**

2.1. Ciclo 5: Modelo HungrySnake 16

2.2. Ciclo 6: Jugar 18

3. Parte III. Bono: Funcionalidades complementarias **21**

3.1. Ciclo 7: Reiniciar 21

3.2. Ciclo 8: Cambiar el tamaño 22

4. Retrospectiva **23**

1. Parte I. Maqueta

1.1. Directorios

El objetivo de este punto es preparar los directorios para el juego HungrySnake.

1. *Preparen un directorio llamado HungrySnake con los directorios src, bin y lib; y los subdirectorios para presentación, dominio y pruebas de unidad. Capturen una pantalla con la estructura.*

   Se creó el directorio `HungrySnake` con las carpetas `src`, `bin` y `lib`. Dentro de `src` están los subdirectorios `domain` (capa de dominio), `presentation` (capa de presentación) y `test` (pruebas de unidad).

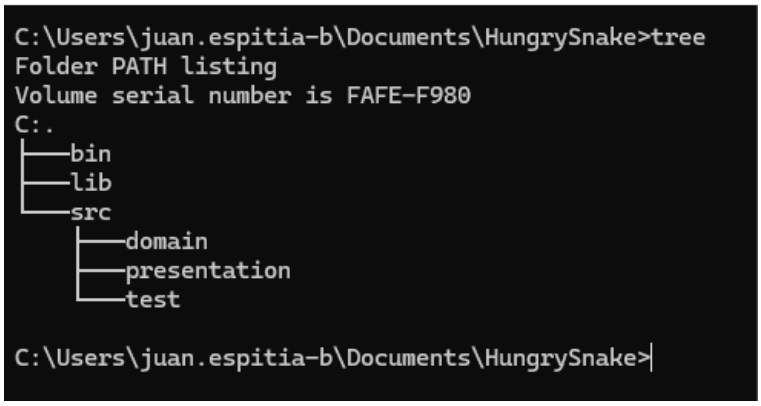


Figura 1: Estructura de directorios del proyecto

   Todo el trabajo se compila y ejecuta desde consola, ubicados en la carpeta `HungrySnake`:

```
javac -d bin src/domain/*.java src/presentation/*.java
java -cp bin presentation.HungrySnakeGUI
```

1.2. Ciclo 0: Ventana vacía – Salir

El objetivo es implementar la ventana principal de HungrySnake con un final adecuado desde el icono de cerrar. Utilizar el esquema de prepareView-prepareController.

1. *Construyan el primer esquema de la ventana de HungrySnake únicamente con el título "HungrySnake". Para esto creen la clase HungrySnakeGUI como un JFrame con su creador (que sólo coloca el título) y el método main que crea un objeto HungrySnakeGUI y lo hace visible. Ejecútenlo. Capturen la pantalla.*

Se creó la clase `HungrySnakeGUI` como subclase de `JFrame`. Su constructor asigna el título y el método `main` crea la ventana y la hace visible con `setVisible(true)`.

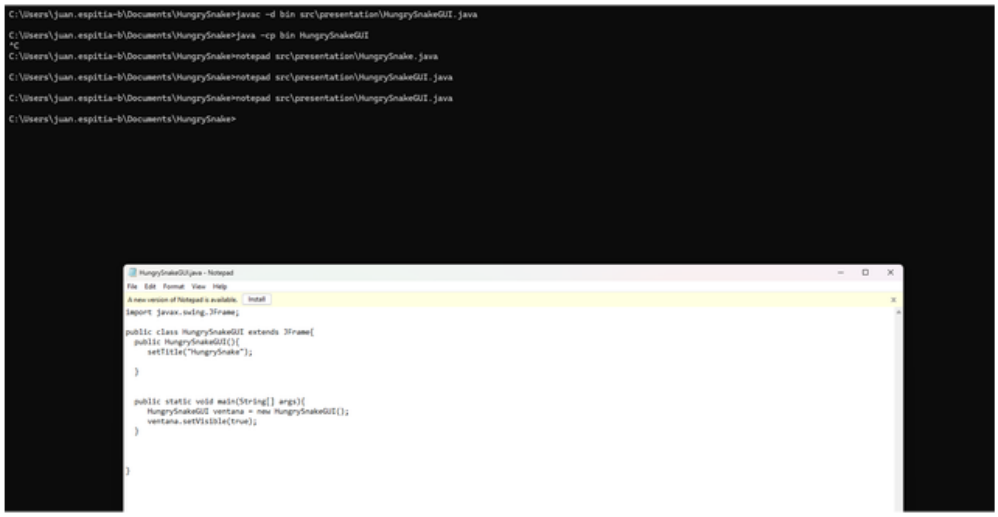


Figura 2: Código inicial y ejecución desde consola



Figura 3: Ventana vacía con el título HungrySnake

2. *Modifiquen el tamaño de la ventana para que ocupe un cuarto de la pantalla y ubíquenla en el centro. Para eso inicien la codificación del método prepareView. Capturen esa pantalla.*

   En `prepareView` se obtiene el tamaño de la pantalla y se asigna a la ventana la mitad del ancho y la mitad del alto, es decir, un cuarto del área total. Luego se centra con `setLocationRelativeTo(null)`.

```
Dimension pantalla = Toolkit.getDefaultToolkit().getScreenSize();
```

```
setSize(pantalla.width / 2, pantalla.height / 2);
setLocationRelativeTo(null);
```

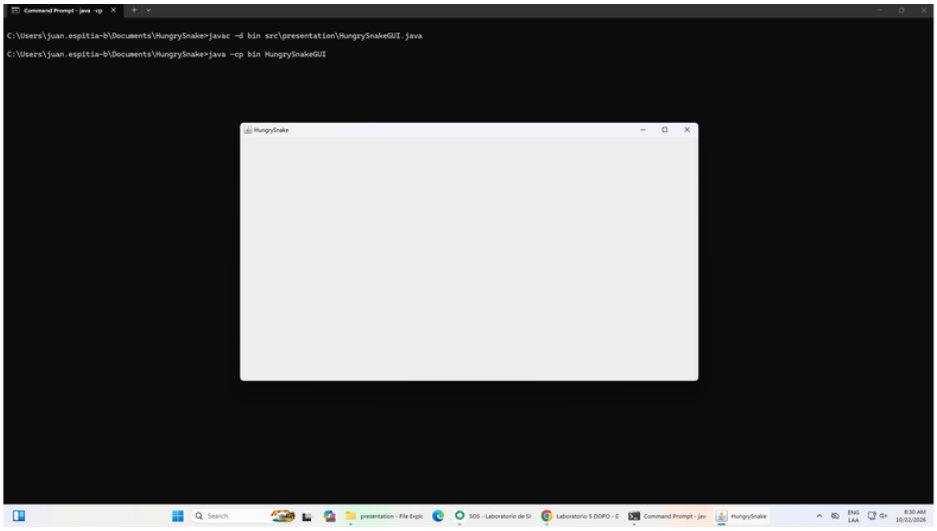


Figura 4: Ventana de un cuarto de pantalla, centrada

3. *Traten de cerrar la ventana. ¿Termina la ejecución? ¿Qué deben hacer en consola para terminar la ejecución?*

   La ventana se cierra; sin embargo, la ejecución no termina. Por defecto un `JFrame` usa `HIDE_ON_CLOSE`, que solo oculta la ventana mientras la máquina virtual sigue en ejecución. La única forma de terminarla es oprimir `Ctrl + C` en la consola.

4. *Estudien en JFrame el método setDefaultCloseOperation. ¿Para qué sirve? ¿Cómo lo usarían si queremos confirmar el cierre de la aplicación? ¿Cómo lo usarían si queremos simplemente cerrar la aplicación?*

   a. Sirve para definir qué ocurre cuando el usuario intenta cerrar un `JFrame`, más específicamente cuando presiona la X de la ventana. Recibe una de las siguientes constantes:

      - `DO_NOTHING_ON_CLOSE`: no hace nada.
      - `HIDE_ON_CLOSE`: oculta la ventana, pero no termina la aplicación (valor por defecto).
      - `DISPOSE_ON_CLOSE`: cierra la ventana y libera sus recursos.
      - `EXIT_ON_CLOSE`: termina la aplicación.

   b. Para confirmar el cierre, se configura con `DO_NOTHING_ON_CLOSE` y se agrega un `WindowListener` que muestre una confirmación antes de cerrar.

   c. Para simplemente cerrar la aplicación, se configura con `EXIT_ON_CLOSE`.

5. *Preparen el "oyente" correspondiente al icono cerrar que le pida al usuario que confirme su selección. Para eso inicien la codificación del método prepareController y el método asociado a la acción (exit). Ejecuten el programa y cierren el programa. Capturen las pantallas.*

```
private void prepareController(){
    setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
    addWindowListener(new WindowAdapter(){
        @Override
        public void windowClosing(WindowEvent e){
            exit();
        }
    });
}

private void exit(){
    int opcion = JOptionPane.showConfirmDialog(this,
        "¿Quiere cerrar la aplicacion?", "Confirmar Salida",
        JOptionPane.YES_NO_OPTION);
    if(opcion == JOptionPane.YES_OPTION){
        System.exit(0);
    }
}
```

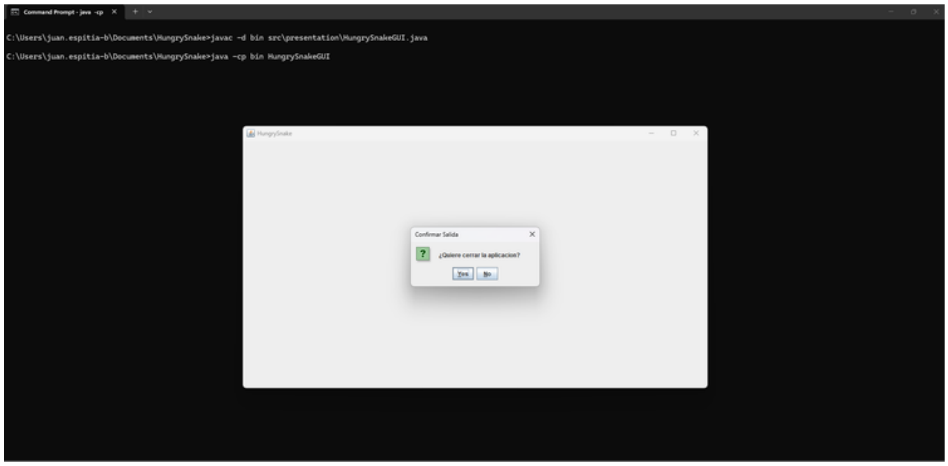


Figura 5: Confirmación al cerrar desde el icono

1.3. Ciclo 1: Ventana con menú – Salir

El objetivo es implementar un menú clásico para la aplicación con un final adecuado desde la opción del menú para salir. El menú debe ofrecer mínimo las siguientes opciones: Nuevo, Abrir – Salvar y Salir. Incluyan los separadores de opciones.

1. *Expliquen los componentes visuales necesarios para este menú. ¿Cuáles serían atributos y cuáles podrían ser variables del método prepareView? Justifique.*

   Componentes visuales necesarios

   - **JMenuBar:** la barra que se ubica en la parte superior del marco del `JFrame`.
   - **JMenu:** los desplegables individuales que se ubican dentro de la barra de menú (en este caso, "Archivo").
   - **JMenuItem:** cada una de las opciones del menú: Nuevo, Abrir, Salvar y Salir.
   - **JSeparator:** la línea horizontal que agrupa visualmente las opciones dentro del menú.

   Atributos de clase

   - **JMenuItem:** requieren oyentes de acción para responder a los clics del usuario. Como los oyentes se asignan en `prepareControllerMenu`, un método distinto de donde se crean, deben ser atributos. Además, durante la ejecución pueden requerir cambios de estado.
   - **JMenuBar:** es opcional como atributo; solo es necesario si se requiere modificar dinámicamente el menú en tiempo de ejecución.

   Variables locales del método

   - **Menú desplegable (JMenu, menuArchivo):** actúa principalmente como contenedor visual. Una vez que se agregan los ítems a `menuArchivo` y este se añade a la barra, no suele requerir interacciones directas adicionales por código.
   - **Separadores (JSeparator):** son elementos estrictamente decorativos para dividir secciones dentro del menú desplegable. No manejan eventos ni cambian de estado.

2. *Construyan la forma del menú propuesto (prepareView - prepareViewMenu). Ejecuten. Capturen la pantalla.*

```
private void prepareViewMenu() {
    menuBar = new JMenuBar();
    JMenu menuArchivo = new JMenu("Archivo");
    itemNuevo = new JMenuItem("Nuevo");
    itemAbrir = new JMenuItem("Abrir");
    itemSalvar = new JMenuItem("Salvar");
    itemSalir = new JMenuItem("Salir");
    menuArchivo.add(itemNuevo);
    menuArchivo.addSeparator();
    menuArchivo.add(itemAbrir);
    menuArchivo.add(itemSalvar);
    menuArchivo.addSeparator();
    menuArchivo.add(itemSalir);
```

```
    menuBar.add(menuArchivo);
    setJMenuBar(menuBar);
}
```

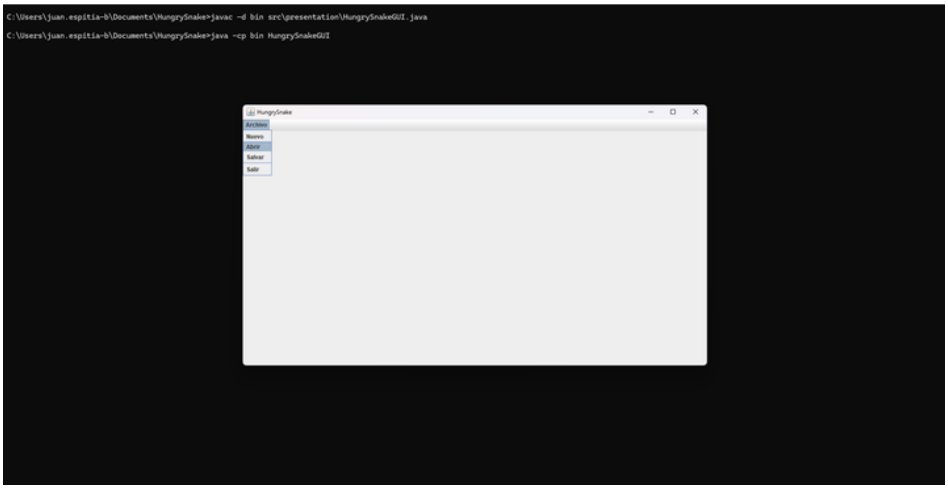


Figura 6: Menú Archivo con sus opciones y separadores

3. *Preparen el "oyente" correspondiente al icono cerrar con confirmación (prepareController - prepareControllerMenu). Ejecuten el programa y salgan del programa. Capturen las pantallas.*

   En `prepareControllerMenu` se asocia a la opción Salir un `ActionListener` anónimo que invoca el mismo método `exit()` del ciclo anterior. De esta forma, salir desde el menú y salir desde el icono de cerrar piden la misma confirmación.

```
itemSalir.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        exit();
    }
});
```

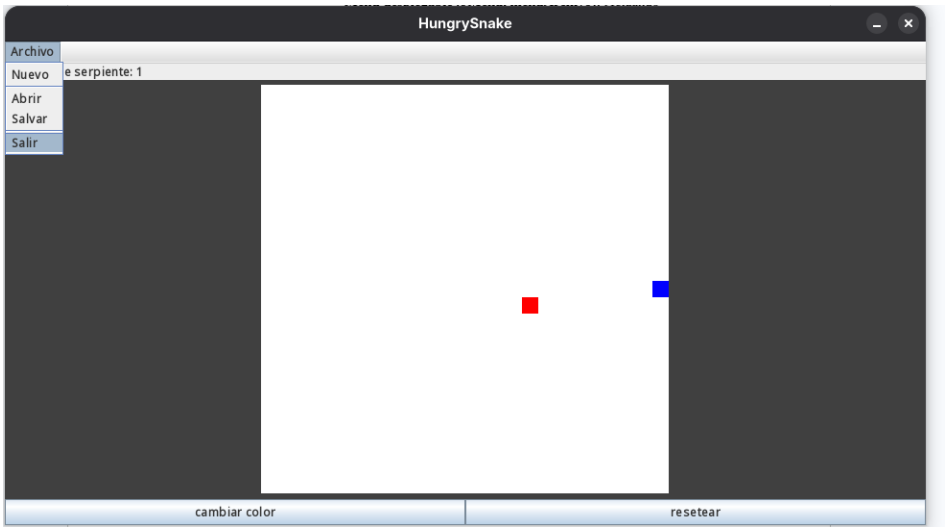


Figura 7: Selección de la opción Salir

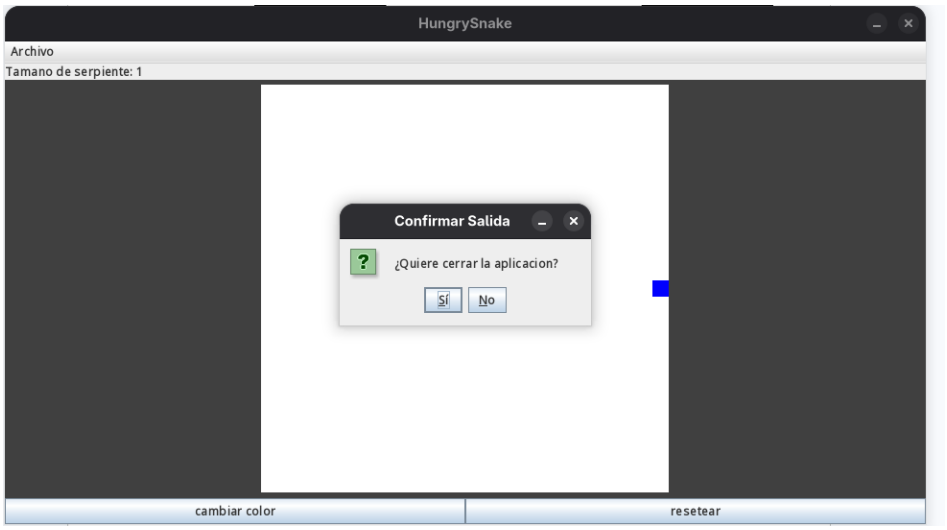


Figura 8: Confirmación de salida desde el menú

1.4. Ciclo 2: Salvar y abrir

El objetivo es preparar la interfaz para las funciones de persistencia.

1. *Detallen el componente JFileChooser, especialmente los métodos: JFileChooser, showOpenDialog, showSaveDialog, getSelectedFile.*

   `JFileChooser` es un componente de Swing que muestra un diálogo estándar para que el usuario navegue por el sistema de archivos y seleccione un archivo.

- **JFileChooser():** constructor que crea el selector de archivos. Se abre en el directorio por defecto del usuario (su carpeta personal).
- **showOpenDialog(Component parent):** muestra un diálogo modal de tipo "Abrir", centrado sobre el componente `parent`. Retorna un entero que indica cómo se cerró el diálogo:
  - `APPROVE_OPTION`: el usuario seleccionó un archivo y aceptó.
  - `CANCEL_OPTION`: el usuario canceló.
  - `ERROR_OPTION`: ocurrió un error o se cerró el diálogo.
- **showSaveDialog(Component parent):** funciona igual que el anterior, pero muestra un diálogo de tipo "Guardar", que permite escribir el nombre de un archivo nuevo. Retorna las mismas constantes.
- **getSelectedFile():** retorna un objeto `java.io.File` con el archivo que el usuario seleccionó. Con `getName()` se obtiene su nombre y con `getAbsolutePath()` su ruta completa.

2. *Implementen parcialmente los elementos necesarios para salvar y abrir. Al seleccionar los archivos indique que las funcionalidades están en construcción detallando la acción y el nombre del archivo seleccionado.*

   Se agregaron oyentes a las opciones Abrir y Salvar, que invocan los métodos `open()` y `save()`. Cada uno crea un `JFileChooser`, muestra el diálogo correspondiente y, si el usuario aprueba la selección, informa con un `JOptionPane` la acción realizada, el nombre del archivo y que la funcionalidad está en construcción.

```
private void open() {
    JFileChooser selector = new JFileChooser();
    int respuesta = selector.showOpenDialog(this);
    if(respuesta == JFileChooser.APPROVE_OPTION) {
        String nombre = selector.getSelectedFile().getName();
        JOptionPane.showMessageDialog(this, "Abrir el archivo "
            + nombre + ": funcionalidad en construccion");
    }
}
```

   El método `save()` es análogo, usando `showSaveDialog`.

   [Pendiente: verificar que el código de open() y save() coincida con el del proyecto**]**

3. *Ejecuten las dos opciones y capturen las pantallas más significativas.*

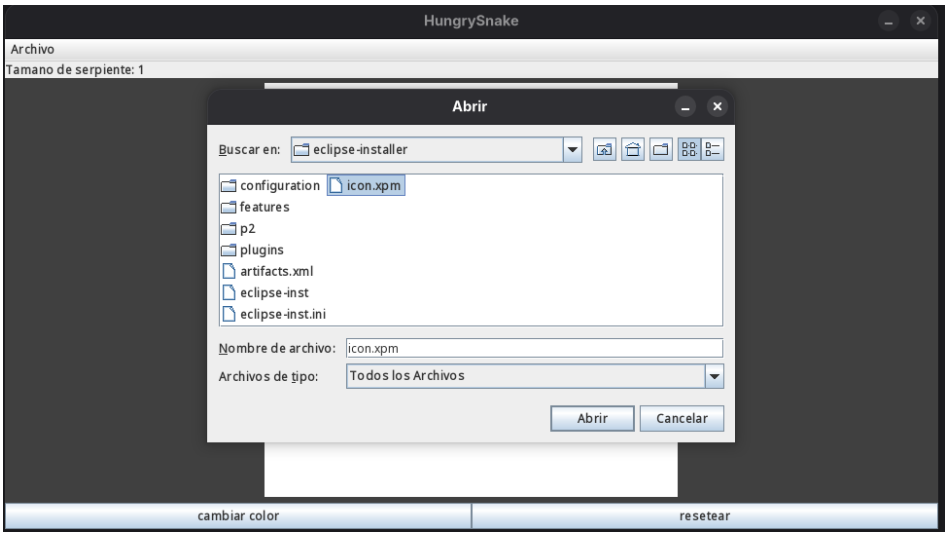


Figura 9: Diálogo para abrir un archivo

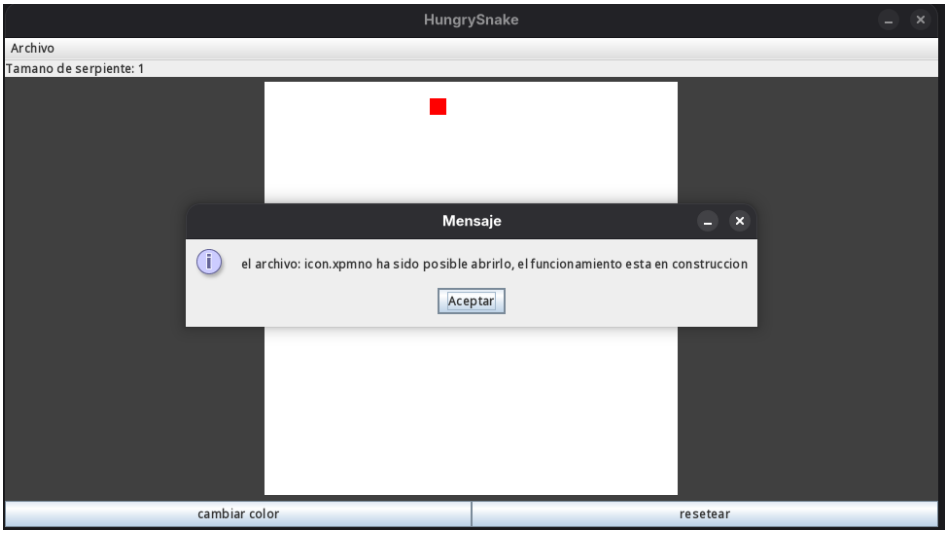


Figura 10: Mensaje de funcionalidad en construcción al abrir

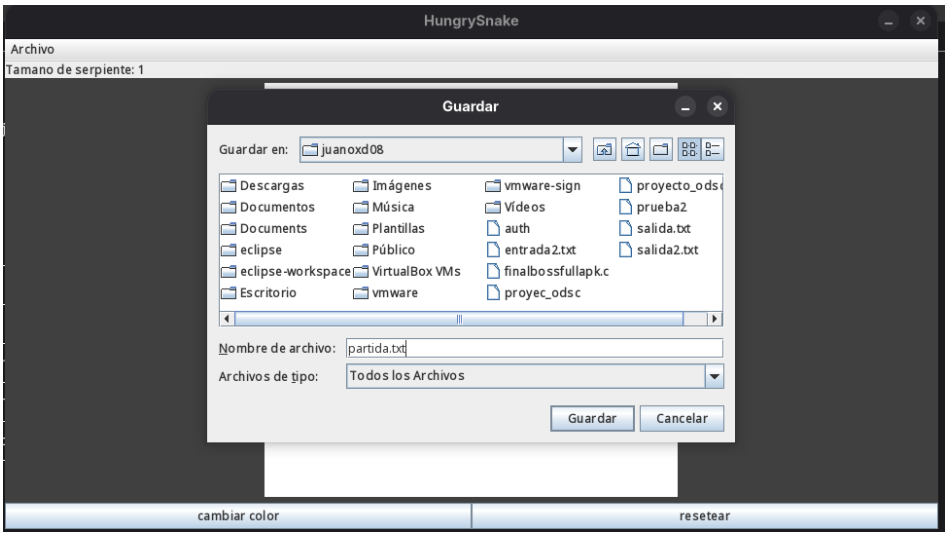


Figura 11: Diálogo para salvar un archivo



Figura 12: Mensaje de funcionalidad en construcción al salvar

1.5. Ciclo 3: Forma de la ventana principal

El objetivo es codificar el diseño de la ventana principal (todos los elementos de primer nivel).

1. *Presenten el bosquejo del diseño de interfaz con todos los componentes necesarios. (Incluyan la imagen)*

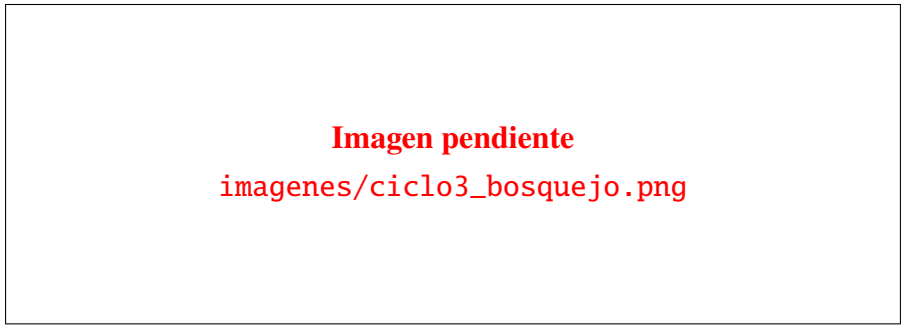


Figura 13: Bosquejo del diseño de la interfaz

La ventana principal usa `BorderLayout`, el layout por defecto del `JFrame`:

- **Barra de menú:** menú Archivo con Nuevo, Abrir, Salvar y Salir.
- **NORTH:** un `JLabel` que informa el tamaño de la serpiente.
- **CENTER:** el tablero de juego.
- **SOUTH:** un `JPanel` con `GridLayout(1, 2)` que contiene los botones "cambiar color" y "resetear".

2. *Continúen con la implementación definiendo los atributos necesarios y extendiendo el método prepareView. Para la zona del tablero definan un método prepareViewBoard y un método refresh que actualiza la vista del tablero considerando, por ahora, el tablero inicial por omisión.*

   Se definieron los atributos `itemInformativo` (`JLabel`), `tablero` (`JPanel`), `colorButton` y `resetButton` (`JButton`), `colorSerpiente` (`Color`) y `game` (el modelo `HungrySnake`).

   - **prepareViewBoard():** crea el tablero y lo ubica en el centro de la ventana.
   - **refresh():** actualiza la vista según el estado del modelo: repinta el tablero y actualiza el texto con el tamaño de la serpiente.

   Decisión de diseño del tablero. La primera versión del tablero usaba un `GridLayout(50, 50)` con 2500 `JPanel`, uno por casilla, y en cada paso cambiaba el color de fondo de cada uno. Esa versión presentó dos problemas:

   1) **Rendimiento:** en cada paso del juego se recorrían y repintaban 2500 componentes, por lo que el juego se veía con tirones.
   2) **Proporción:** el `GridLayout` estira las celdas para llenar todo el espacio disponible. Como la ventana es más ancha que alta, las celdas quedaban rectangulares y la serpiente se veía más delgada al moverse en horizontal que en vertical.

   Por esta razón, el tablero se reemplazó por un único `JPanel` que se dibuja a sí mismo sobrescribiendo `paintComponent(Graphics g)`:

- El tamaño de cada celda se calcula como `Math.min(ancho, alto) / n`, donde `n` es el lado del tablero. Así las celdas siempre son cuadradas.
- El tablero se centra con los desplazamientos `x0 = (ancho - celda*n) / 2` y `y0 = (alto - celda*n) / 2`.
- Cada elemento (cuerpo, fruta y obstáculo) se pinta con `g.fillRect(x0 + columna*celda, y0 + fila*celda, celda, celda)`.
- El método `refresh()` solo necesita llamar a `tablero.repaint()`.

```
private void prepareViewBoard() {
    tablero = new JPanel() {
        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            int n = game.getTamanoTablero();
            int celda = Math.min(getWidth(), getHeight()) / n;
            int x0 = (getWidth() - celda * n) / 2;
            int y0 = (getHeight() - celda * n) / 2;

            g.setColor(Color.WHITE);
            g.fillRect(x0, y0, celda * n, celda * n);

            g.setColor(colorSerpiente);
            for (int[] p : game.getCulebra())
                g.fillRect(x0 + p[1] * celda, y0 + p[0] * celda, celda, celda);

            g.setColor(Color.RED);
            for (int[] p : game.getFrutas())
                g.fillRect(x0 + p[1] * celda, y0 + p[0] * celda, celda, celda);

            g.setColor(Color.BLACK);
            for (int[] p : game.getObstaculos())
                g.fillRect(x0 + p[1] * celda, y0 + p[0] * celda, celda, celda);

            Toolkit.getDefaultToolkit().sync();
        }
    };
    tablero.setBackground(Color.DARK_GRAY);
    add(tablero, BorderLayout.CENTER);
}
```

3. *Ejecuten y capturen la pantalla.*

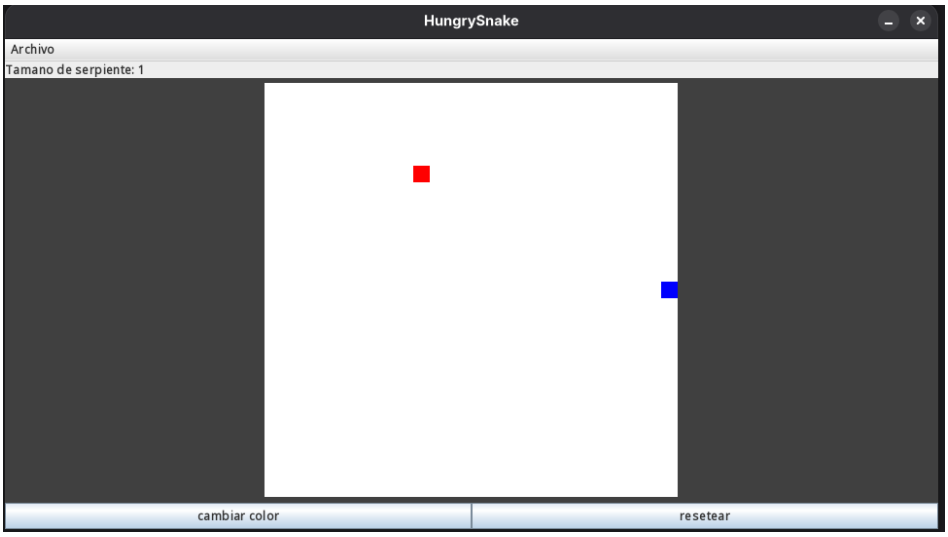


Figura 14: Ventana principal con todos sus componentes

1.6. Ciclo 4: Cambiar colores

El objetivo es implementar este caso de uso.

1. *Expliquen los elementos (vista – controlador) necesarios para implementar este caso de uso.*

   Vista

   - El botón `colorButton` (“cambiar color”).
   - El atributo `colorSerpiente`, que guarda el color con el que se pinta la serpiente.
   - El diálogo `JColorChooser`, donde el usuario elige el color.

   Controlador

   - Un `ActionListener` anónimo asociado a `colorButton` que invoca el método `changeColor()`.
   - El método `changeColor()`, que abre el `JColorChooser` y, si el usuario elige un color, actualiza `colorSerpiente` y llama a `refresh()` para repintar el tablero.

   El color es una decisión de presentación y no del dominio, por lo que se guarda en la interfaz y no en el modelo `HungrySnake`.

2. *Detalle el comportamiento de JColorChooser, especialmente el método estático showDialog.*

   `JColorChooser` es un componente de Swing que presenta una paleta para seleccionar colores. Su método más usado es:

```
static Color showDialog(Component component, String title, Color initialColor)
```

Al ser estático, no es necesario crear un objeto `JColorChooser`. El método muestra un diálogo modal centrado sobre `component`, con el título `title` y con `initialColor` seleccionado inicialmente. Retorna el `Color` elegido si el usuario presiona Aceptar, o `null` si cancela o cierra el diálogo. Por eso es necesario verificar si el resultado es `null` antes de usarlo.

3. *Implementen los componentes necesarios para cambiar los colores requeridos.*

```
colorButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        changeColor();
    }
});

private void changeColor() {
    Color temp = JColorChooser.showDialog(this,
        "cambio de color de la serpiente", colorSerpiente);
    if(temp == null) {
        return;
    }
    colorSerpiente = temp;
    refresh();
}
```

Al botón se le asignó `setFocusable(false)` para que, después de usarlo, el teclado siga controlando la serpiente.

4. *Ejecuten el caso de uso y capturen las pantallas más significativas.*

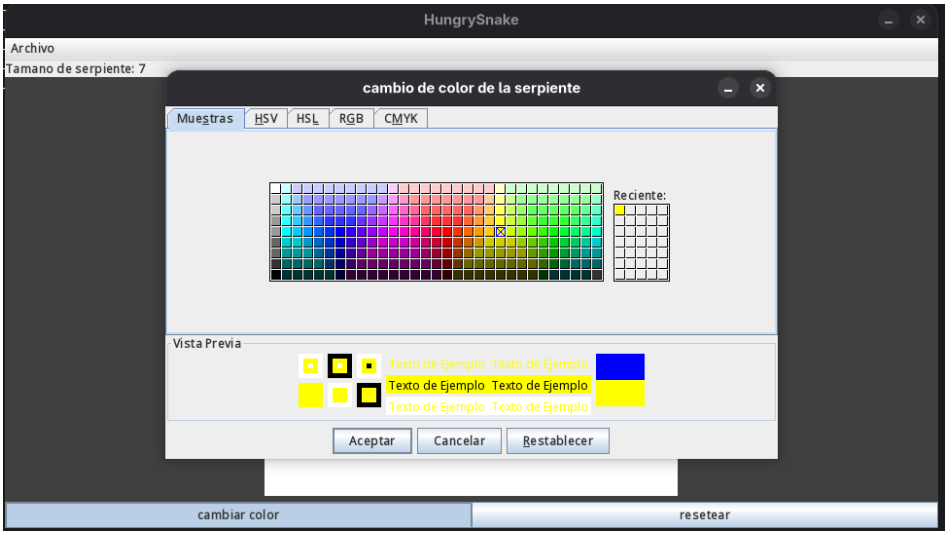


Figura 15: Diálogo JColorChooser

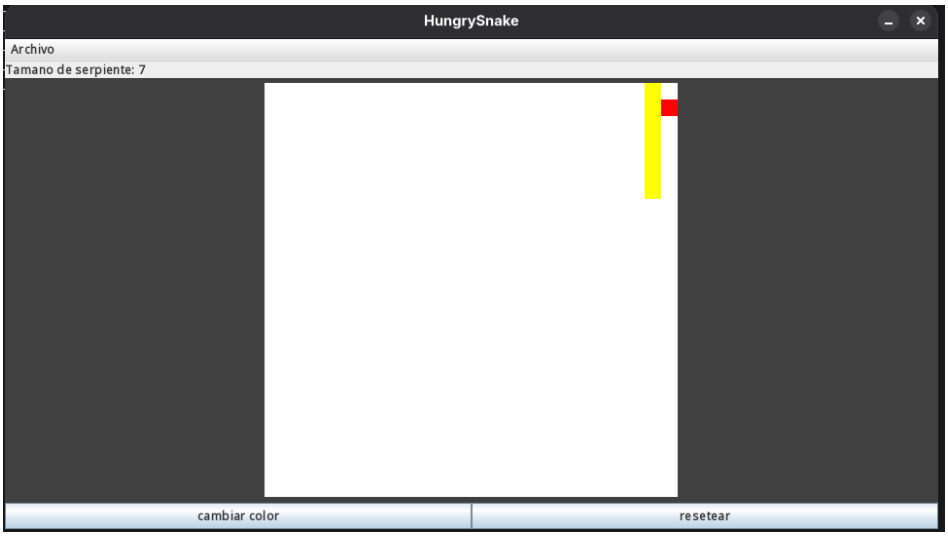


Figura 16: Serpiente con el nuevo color

2. Parte II. Funcionalidades básicas

2.1. Ciclo 5: Modelo HungrySnake

El objetivo es implementar la capa de dominio para HungrySnake.

1. *Construya los métodos básicos del juego.*

   La clase `domain.HungrySnake` representa el estado del juego. El tablero se modela como una cuadrícula de 25×25 casillas, donde cada posición es un arreglo `{fila, columna}`.

Atributo	**Descripción**
`tamanoTablero`	Lado del tablero (25).
`culebra`	Posiciones del cuerpo; la posición 0 es la cabeza.
`frutas`	Posiciones de las frutas.
`obstaculos`	Posiciones de los obstáculos.
`actualDirection`	Dirección actual del movimiento (W, A, S, D).
`inGame`	Indica si el juego sigue activo.

Tabla 1: Atributos de la clase HungrySnake

Método	Responsabilidad
`HungrySnake()`	Crea la serpiente en el centro del tablero y genera la primera fruta.
`move(char)`	Avanza la serpiente una casilla. Ignora la media vuelta; si come una fruta crece y, con 10 % de probabilidad, aparece un obstáculo; si choca con una pared, un obstáculo o su propio cuerpo, termina el juego.
`isValid(f, c)`	Indica si una casilla está dentro del tablero y libre.
`generarFrutas()`	Ubica una fruta en una casilla libre aleatoria.
`generarObstaculos()`	Ubica un obstáculo en una casilla libre aleatoria.
`getCulebra()`, `getFrutas()`, `getObstaculos()`	Consultas de las posiciones para la vista.
`isInGame()`, `getTamano()`, `getTamanoTablero()`	Consultas del estado del juego.

Tabla 2: Métodos de la clase HungrySnake

MDD. El diseño del dominio se modeló en el diagrama de clases `hungrySnake.astah`.



Figura 17: Diagrama de clases del dominio

BDD. Las pruebas de unidad, solo de la capa de dominio, se escribieron a partir de los comportamientos esperados del juego:

Prueba	Comportamiento esperado
`shouldStartWithSizeOne`	La serpiente empieza con tamaño 1 y el juego activo.
`shouldStartInTheCenter`	La cabeza empieza en la posición {12, 12}.
`shouldStartWithOneFruit`	Al iniciar hay exactamente una fruta.
`shouldMoveRight`	Al mover a la derecha la cabeza pasa a {12, 13} sin cambiar el tamaño.
`shouldNotTurnBack`	Si se mueve a la derecha, un movimiento a la izquierda mantiene la dirección.
`shouldDieWhenHittingWall`	Al salir del tablero el juego termina.
`shouldNotMoveWhenDead`	Después de perder, la serpiente no cambia de posición.
`shouldIgnoreInvalidKey`	Una tecla distinta de W, A, S o D no cambia nada.
`shouldAcceptLowercase`	Las teclas en minúscula funcionan igual que en mayúscula.
`shouldPlaceFruitInsideBoard`	Toda fruta queda dentro del tablero.
`shouldGrowWhenEatingFruit`	Al llegar a la fruta el tamaño pasa a 2.

Tabla 3: Pruebas de unidad del dominio

[Pendiente: ajustar la tabla a las pruebas que finalmente se implementen**]**

2. *Ejecuten las pruebas y capturen el resultado.*

```
javac -d bin -cp lib/junit-platform-console-standalone-1.10.2.jar src/domain/*.java
    src/presentation/*.java src/test/*.java
java -jar lib/junit-platform-console-standalone-1.10.2.jar -cp bin --scan-classpath
```

Imagen pendiente

imagenes/ciclo5_resultado_pruebas.png

Figura 18: Resultado de la ejecución de las pruebas

2.2. Ciclo 6: Jugar

El objetivo es implementar el juego.

1. *Expliquen los elementos necesarios para implementar este caso de uso.*

   - **Modelo:** el método `move(char)` de `HungrySnake`, que contiene toda la lógica de un paso del juego.

- **Vista:** el tablero, que dibuja el estado del modelo en `paintComponent`, y el `JLabel` con el tamaño de la serpiente.
- **Controlador:**
  - Un `KeyListener` (`KeyAdapter` anónimo) sobre la ventana, que guarda en el atributo `ultimaTecla` la última tecla válida (W, A, S o D).
  - Un `javax.swing.Timer` que cada 55 ms ejecuta `game.move(ultimaTecla)` y luego `refresh()`.
  - Cuando el modelo indica que el juego terminó (`isInGame()` es falso), `refresh()` detiene el `Timer` y muestra un `JOptionPane` avisando que el jugador perdió.

2. *Implementen los componentes necesarios para jugar. ¿Cuántos oyentes necesitan? ¿Por qué?*

```
addKeyListener(new KeyAdapter () {
    @Override
    public void keyPressed(KeyEvent k) {
        char tecla = Character.toUpperCase(k.getKeyChar());
        if (tecla == 'W' || tecla == 'A' || tecla == 'S' || tecla == 'D') {
            ultimaTecla = tecla;
        }
    }
});

timer = new Timer(55, new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        game.move(ultimaTecla);
        refresh();
    }
});
timer.start();
```

Para jugar se necesitan **dos oyentes**:

1) **KeyListener:** atiende los eventos del usuario. Solo registra la dirección deseada; no mueve la serpiente.
2) **ActionListener del Timer:** atiende los eventos del tiempo. En cada intervalo avanza la serpiente un paso y refresca la vista.

Se necesitan dos porque son dos fuentes de eventos distintas. En HungrySnake la serpiente debe seguir avanzando aunque el usuario no presione ninguna tecla, y eso solo lo puede producir un evento periódico, como el del `Timer`. Si el movimiento dependiera solo del teclado, la serpiente avanzaría únicamente al presionar teclas. Separarlos también evita que presionar teclas muy rápido acelere la serpiente: la velocidad la fija únicamente el `Timer`.

3. *Ejecuten el caso de uso y capturen las pantallas más significativas.*



Figura 19: Inicio del juego



Figura 20: La serpiente después de comer varias frutas

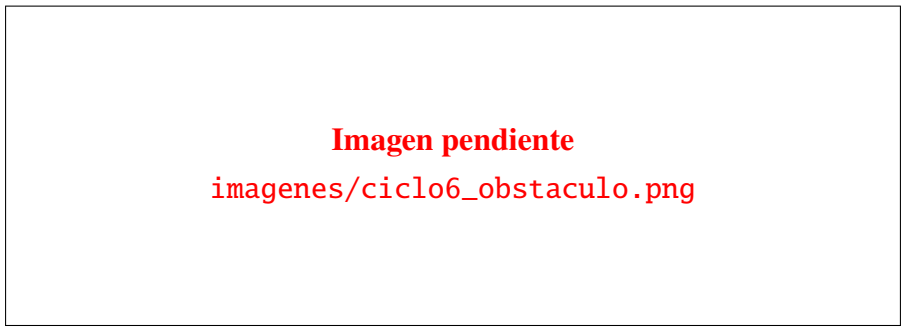


Figura 21: Aparición de obstáculos en el tablero



Figura 22: Mensaje al perder el juego

3. Parte III. Bono: Funcionalidades complementarias

3.1. Ciclo 7: Reiniciar

El objetivo es implementar este caso de uso.

1. *Expliquen los elementos a usar para implementar este caso de uso.*

   [Pendiente: completar según la implementación**]**

2. *Implementen los elementos necesarios para reiniciar.*

   [Pendiente: código de la implementación**]**

3. *Ejecuten el caso de uso y capturen las pantallas más significativas.*



Figura 23: Juego antes de reiniciar



Figura 24: Juego después de reiniciar

3.2. Ciclo 8: Cambiar el tamaño

El objetivo es implementar este caso de uso.

1. *Expliquen los elementos a usar para implementar este caso de uso.*

   [Pendiente: completar según la implementación**]**

2. *Implementen los elementos necesarios para cambiar el tamaño del juego.*

   [Pendiente: código de la implementación**]**

3. *Ejecuten el caso de uso y capturen las pantallas más significativas.*



Figura 25: Solicitud del nuevo tamaño

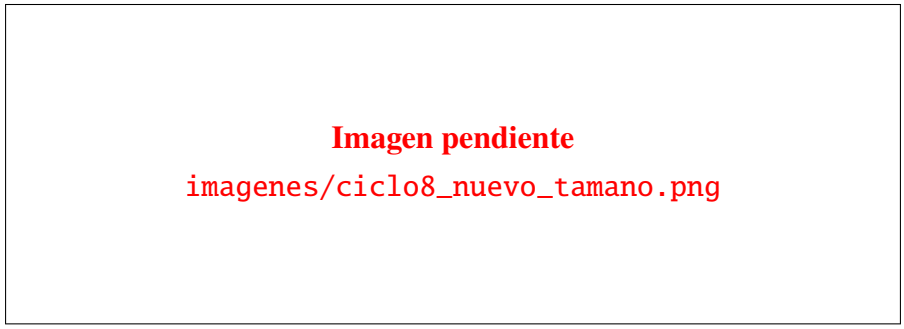


Figura 26: Tablero con el nuevo tamaño

4. Retrospectiva

1. *¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)*

   - Juan David Espitia: **[Pendiente:** horas**]**
   - Cristian Camilo Salamanca: **[Pendiente:** horas**]**

2. *¿Cuál es el estado actual del laboratorio? ¿Por qué?*

   Las Partes I y II están implementadas: la ventana con cierre confirmado, el menú, las opciones de abrir y salvar en construcción, el cambio de color, el modelo del juego y el caso de uso jugar. **[Pendiente:** indicar el estado de las pruebas y de los bonos**]**

3. *Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿Por qué?*

   [Pendiente: revisar y redactar con sus palabras**]** La práctica *"When a bug is found tests are created"*. Cuando se encontraba un comportamiento incorrecto en el modelo, escribir primero una prueba que lo reprodujera permitía verificar la corrección y asegurar que el error no volviera a aparecer.

4. *¿Cuál consideran fue el mayor logro? ¿Por qué?*

   [Pendiente: revisar y redactar con sus palabras**]** Separar correctamente el modelo (`domain`) de la vista-controlador (`presentation`). Gracias a esto, toda la lógica del juego se puede probar sin abrir ninguna ventana, y la forma de dibujar el tablero se pudo cambiar por completo sin modificar el modelo.

5. *¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?*

   [Pendiente: revisar y redactar con sus palabras**]** El rendimiento y la proporción del tablero. Con 2500 `JPanel` en un `GridLayout`, el juego se veía con tirones y las celdas se estiraban según la forma de la ventana. Se resolvió dibujando el tablero en un solo

`JPanel` con `paintComponent`: las celdas se calculan cuadradas, el tablero se centra y se usa `Toolkit.getDefaultToolkit().sync()` para que el dibujo sea fluido.

6. *¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?*

   [Pendiente: respuesta del equipo**]**

7. *¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.*

   [Pendiente: dejar solo las referencias que realmente usaron e indicar la más útil**]**

- Oracle. (s.f.). *JFrame (Java SE 21 & JDK 21)*. `https://docs.oracle.com/en/java/javase/21/docs/api/java.desktop/javax/swing/JFrame.html`
- Oracle. (s.f.). *JFileChooser (Java SE 21 & JDK 21)*. `https://docs.oracle.com/en/java/javase/21/docs/api/java.desktop/javax/swing/JFileChooser.html`
- Oracle. (s.f.). *JColorChooser (Java SE 21 & JDK 21)*. `https://docs.oracle.com/en/java/javase/21/docs/api/java.desktop/javax/swing/JColorChooser.html`
- Oracle. (s.f.). *Timer (Java SE 21 & JDK 21)*. `https://docs.oracle.com/en/java/javase/21/docs/api/java.desktop/javax/swing/Timer.html`
- Oracle. (s.f.). *Performing Custom Painting*. The Java Tutorials. `https://docs.oracle.com/javase/tutorial/uiswing/painting/`
- Oracle. (s.f.). *How to Use File Choosers*. The Java Tutorials. `https://docs.oracle.com/javase/tutorial/uiswing/components/filechooser.html`